Itchy Industries

Delivering Contoso's AWS landing zone

Infrastructure as Code: options and recommendation

Rudolf Kreibich, Head of Cloud Engineering | Itchy Industries

One foundation before departments move in

Consistent baselines	Every new account gets the approved security and logging setup.
Faster onboarding	A new department account in hours instead of weeks.
Reviewed changes	Every change is a pull request with a named approver.
Rebuildable	The whole foundation can be recreated from code.
Less drift	Manual changes are detected and corrected.

Without it, each department builds AWS its own way: inconsistent security, slow onboarding and no audit trail.

The landing zone: accounts and guardrails

Management account	Billing, AWS Organizations and Control Tower
Security OU	Log Archive for central logs, Audit for security tooling
Infrastructure OU	Network for shared connectivity, Shared Services for CI/CD
Workloads OU	One OU per department, each with dev, test and prod accounts
Sandbox OU	Experiments, with spend limits

Guardrails attach to OUs, so a new account inherits them on day one. The OU layout is a Contoso design choice.

COMPARISON

Three ways to write it, two engines to run it

	Terraform	CloudFormation	AWS CDK
You write	HCL	YAML / JSON	TypeScript, Python…
Runs on	Terraform + providers	AWS CloudFormation	Generated CloudFormation
State	You run it	Managed by AWS	Managed by AWS
Many accounts	Account Factory (AFT)	StackSets, CfCT	Via CloudFormation; LZA
Covers	AWS + other providers	AWS only	AWS only
Support	HashiCorp, community	AWS Support	AWS, open source

Terraform: one tool across providers

Pros

One workflow for AWS and other tools

Readable plan before every change

Large module ecosystem; AWS-built AFT

Cons

State to run, lock and protect

Provider upgrades need testing

BSL licence since 2023 (OpenTofu fork)

Prerequisites

HCL skills, a state backend with locking,
a CI runner and pinned provider versions

CloudFormation: nothing extra to operate

Pros

Nothing to operate: AWS runs state

StackSets reach every new account

Native to Control Tower; AWS Support

Cons

AWS only

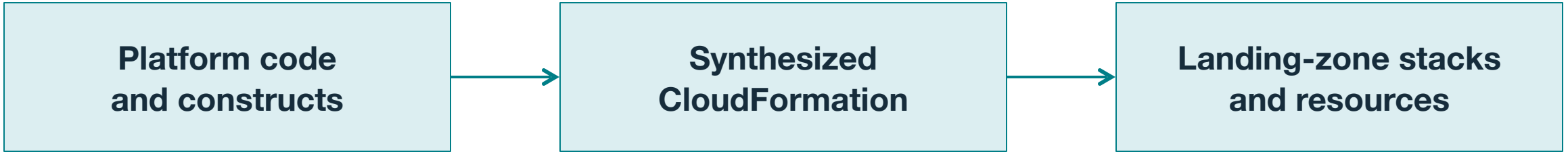
Long YAML, limited logic

Replacements need careful review

Prerequisites

YAML and AWS / IAM skills, StackSets trusted access, a pipeline with cfn-lint and cfn-guard checks

CDK: infrastructure as real code



Pros

Loops, reuse and unit tests in code
Deploys through CloudFormation
AWS built Landing Zone Accelerator on it

Cons

Two layers to debug: code + template
Fast-moving library updates
Needs software engineers

Prerequisites

TypeScript or Python skills, Node.js and the CDK CLI, bootstrap in every account and Region

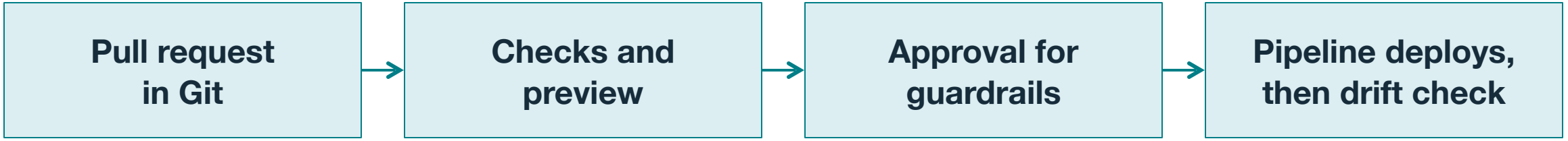
RECOMMENDATION

CloudFormation on Control Tower

Nothing to run	**AWS runs state and rollback**	AWS Support covers it
Many accounts	**StackSets reach new accounts**	Automatically, per OU
Control Tower fit	**AWS's documented path**	CfCT and CT resources
Easy to staff	**YAML plus AWS knowledge**	No state or code skills

Otherwise: Terraform + AFT if a strong Terraform team exists; add LZA for strict compliance. Application teams keep their own tools.

How a change reaches AWS



Who owns what

Platform: code, pipeline, operations
Security: policies and guardrails
Control Tower: its own resources

Controlled access

Pipeline deploys, not people
Short-lived deployment credentials
Security approves guardrail changes

Roadmap and decisions needed

Roadmap

1. Foundation: Control Tower, SSO, logs
2. Shared services: network, pipeline
3. Pilot: first department live
4. Scale: remaining departments

Decisions needed

Account and OU structure
Identity source for SSO
Regions and data residency
Platform owner + 2–3 engineers

Milestone

First department live in its own dev, test and prod accounts, created entirely from code.